

Homework for Search

2026 Fall

Overview

Due October 23, 23:59, China Standard Time

This homework includes written and programming problems related to Lecture 2.

- For **writing component**, please write your answers in a **report pdf** file. You can use $\text{\LaTeX}$, convert from Word, from notebook apps on tablets, scan your handwritten documents, and etc. Your derivation and proof should be completed in the **pdf** file. Besides, your section titles and subsection titles should be the number of question. i.e., # Q1 ## Q1.1.
- For **programming component**, please submit your code in **search** project folder. Note that your analysis and some requested results should be included in the same report **pdf** file as the writing component.
- **AI Agent Policy**: AI agents are prohibited except for the competition part of the programming component 2.

Please organize your submitted files as follows:

```
report_${student_id}.pdf
search
|--...
```

Finally, please zip the `report_${student_id}.pdf` file and `search` directory (rather than their parent directory) into `${student_id}.zip`. You should submit your `${student_id}.zip` to Web Learner before **October 23, 23:59, China Standard Time**.

1 Writing Component (40 pts)

1.1 Search with Heuristics (15 pts)

Imagine an $n \times n$ grid where there is a car in each of cells $(1, 1)$ to $(n, 1)$ —the bottom row. Every time step, each car can move up, down, left, right, or stay put. If a car does not move, at most one other car can hop over it. At most one car can be in a cell at any given time. Cars move simultaneously (so a car can move into the place of another car if the other car moves out), and two cars are not allowed to move into the same cell. Moving a car in any direction (including hopping over another car) incurs a cost of 1, whereas staying put has no cost. The total cost at any time step is the sum of the cost for each car. The goal is to move all the cars to the top row but in reverse order so that the car starting in cell $(i, 1)$ ends up in cell $(n - i + 1, n)$.

(a) What is the size of the state space in big- $\mathcal{O}$?

(b) Suppose there was only one car starting at $(i, 1)$, and the problem was to only move that car to its goal location. Consider the following heuristic for moving car i from cell (x_i, y_i) to its goal at $(n - i + 1, n)$: $h_i(x_i, y_i) = |(n - i + 1) - x_i| + |n - y_i|$. Is it an admissible heuristic? Would it be admissible when the other $n - 1$ cars are also at various positions on the grid?

(c) Which of the following are admissible heuristics for the original problem and why? (Note: h_i is the heuristic in the previous part.)

- (i) $\sum_i h_i$ (ii) $\min(h_1, \dots, h_n)$ (iii) $\max(h_1, \dots, h_n)$
- (iv) $n \min(h_1, \dots, h_n)$ (v) $n \max(h_1, \dots, h_n)$

1.2 Adjusted Heuristics (10 pts)

The consistent (or monotone) condition on the heuristic function requires that $h(n_i) \leq h(n_j) + c(n_i, n_j)$ for all node-successor pairs (n_i, n_j) , where $c(n_i, n_j)$ is the cost on the arc from n_i to n_j . It has been suggested that when the consistent condition is not satisfied, we can adjust h during the search to satisfy the condition. The idea is that whenever a node n_i is expanded, with successor node n_j , we can increase $h(n_j)$ by whatever amount is needed to satisfy the consistent condition. Construct an example to show that even with this scheme, when a node is expanded, we have not necessarily found the least costly path to it.

1.3 Adversarial Search (15 pts)

Table 1: Normal form of the game.

		MIN Strategy			
		(C, C)	(C, D)	(D, C)	(D, D)
MAX Strategy	$A; (E, E)$	12	12	9	9
	$A; (E, F)$	12	12	7	7
	$A; (F, E)$	5	5	9	9
	$A; (F, F)$	5	5	7	7
	$B; (E, E)$	8	3	8	3
	$B; (E, F)$	8	15	8	15
	$B; (F, E)$	2	3	2	3
	$B; (F, F)$	2	15	2	15

Table 1 shows the normal form of a two-player, zero-sum game. The game is played for three rounds, in the order of MAX, MIN, MAX. The pure strategy $P_{MAX} = A; (E, F)$ means the MAX player chooses action A at root, and will choose action E if the MIN player chooses action C , F if the MIN player chooses action D . $P_{MIN} = (C, D)$ means the MIN player will choose action C if the MAX player chooses action A in the first round, D if the MAX player chooses action B in the first round.

(a) Convert the normal form of the game to a game tree.

- (b) Find the minimax value of the game tree's root node.
- (c) Apply alpha-beta pruning to the game tree. Which branch is pruned?

2 Programming Component 1: Search in Pacman (30 pts)

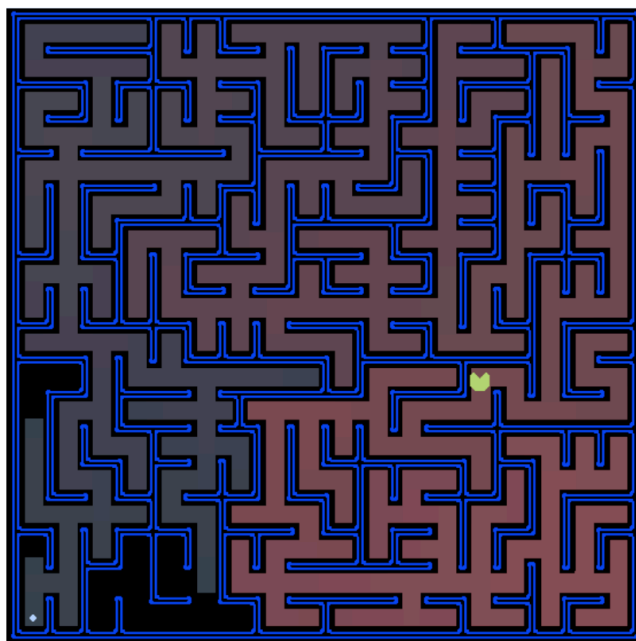


Figure 1: Pacman environments.

2.1 Introduction

In this project, your Pacman agent will find paths through his maze world to reach a particular location and collect food efficiently. You will build general search algorithms and apply them to Pacman scenarios.

This project includes an autograder for you to grade your answers on your machine. This can be run with the command:

```
python autograder.py
```

You can find all the code and supporting files in **search** folder. The code for this project consists of several Python files, some of which you will need to read and understand in order to complete the assignment, and some of which you can ignore.

Files you'll edit:

`search.py` Where all of your search algorithms will reside.

`searchAgents.py` Where all of your search-based agents will reside.

Files you might want to look at:

`pacman.py` The main file that runs Pacman games. This file describes a Pacman GameState type you use in this project.

`game.py` The logic behind how the Pacman world works. This file describes several supporting types like AgentState, Agent, Direction, and Grid.

`util.py` Useful data structures for implementing search algorithms.

Supporting files you can ignore:

`graphicsDisplay.py` Graphics for Pacman

`graphicsUtils.py` Support for Pacman graphics

`textDisplay.py` ASCII graphics for Pacman

`ghostAgents.py` Agents to control ghosts

`keyboardAgents.py` Keyboard interfaces to control Pacman

`layout.py` Code for reading layout files and storing their contents Project autograder

`autograder.py` Parses autograder test and solution files

`testParser.py` General autograding test classes

`testClasses.py` Directory containing the test cases for each question

`test_cases/ searchTestClasses.py` Project 1 specific autograding test classes

Files to Edit: You will fill in portions of `search.py` and `searchAgents.py` during the assignment. Please do not change the other files in this distribution.

Evaluation: Your code will be autograded for technical correctness. Please do not change the names of any provided functions or classes within the code, or you will wreak havoc on the autograder. However, the correctness of your implementation – not the autograder’s judgments – will be the final judge of your score. If necessary, we will review and grade assignments individually to ensure that you receive due credit for your work.

2.2 Welcome to Pacman

Changing to the directory of `search`, you should be able to play a game of Pacman by typing the following at the command line:

```
python pacman.py
```

Pacman lives in a shiny blue world of twisting corridors, and tasty round treats. Navigating this world efficiently will be Pacman's first step in mastering his domain.

The simplest agent in `searchAgents.py` is called the `GoWestAgent`, which always goes West (a trivial reflex agent). This agent can occasionally win:

```
python pacman.py --layout testMaze --pacman GoWestAgent
```

But, things get ugly for this agent when turning is required:

```
python pacman.py --layout tinyMaze --pacman GoWestAgent
```

If Pacman gets stuck, you can exit the game by typing CTRL-c into your terminal.

Soon, your agent will solve not only `tinyMaze`, but any maze you want.

Note that `pacman.py` supports a number of options that can each be expressed in a long way (e.g., `--layout`) or a short way (e.g., `-l`). You can see the list of all options and their default values via:

```
python pacman.py -h
```

Also, all of the commands that appear in this project also appear in `commands.txt` for easy copying and pasting. In UNIX/Mac OS X, you can even run all these commands in order with `bash commands.txt`.

Note: if you get error messages regarding Tkinter, see [this page](#).

2.3 Question 1 (2 pts): Finding a Fixed Food Dot using Depth First Search

In `searchAgents.py`, you'll find a fully implemented `SearchAgent`, which plans out a path through Pacman's world and then executes that path step-by-step. The search algorithms for formulating a plan are not implemented – that's your job.

First, test that the `SearchAgent` is working correctly by running:

```
python pacman.py -l tinyMaze -p SearchAgent -a fn=tinyMazeSearch
```

The command above tells the `SearchAgent` to use `tinyMazeSearch` as its search algorithm, which is implemented in `search.py`. Pacman should navigate the maze successfully.

Now it's time to write full-fledged generic search functions to help Pacman plan routes! The pseudocode for the search algorithms you'll write can be found in the lecture slides. Remember that a search node must contain a state and the information necessary to reconstruct the path (plan) that gets to that state.

Important note: All of your search functions need to return a list of actions that will lead the agent from the start to the goal. These actions all have to be legal moves (valid directions, no moving through walls).

Important note: Make sure to use the `Stack`, `Queue` and `PriorityQueue` data structures provided to you in `util.py`! These data structure implementations have particular properties

which are required for compatibility with the autograder.

Hint: Each algorithm is very similar. Algorithms for DFS, BFS, UCS, and A* differ only in the details of how the fringe is managed. So, concentrate on getting DFS right; the rest should be relatively straightforward. Indeed, one possible implementation requires only a single generic search method which is configured with an algorithm-specific queuing strategy. (Your implementation need not be of this form to receive full credit).

Implement the depth-first search (DFS) algorithm in the `depthFirstSearch` function in `search.py`. To make your algorithm complete, write the graph search version of DFS, which avoids expanding any already visited states.

Your code should quickly find a solution for:

```
python pacman.py -l tinyMaze -p SearchAgent
```

The Pacman board will show an overlay of the states explored and the order in which they were explored (brighter red means earlier exploration).

Please run the below command to see if your implementation passes all the autograder test cases.

```
python autograder.py -q q1
```

The screenshot of the output should be included in the report pdf file.

2.4 Question 2 (2 pts): Breadth-First Search

Implement the breadth-first search (BFS) algorithm in the `breadthFirstSearch` function in `search.py`. Again, write a graph search algorithm that avoids expanding any already visited states. Test your code the same way you did for the depth-first search.

```
python pacman.py -l mediumMaze -p SearchAgent -a fn=bfs
python pacman.py -l bigMaze -p SearchAgent -a fn=bfs -z .5
```

Does BFS find a least-cost solution? If not, check your implementation. Hint: If Pacman moves too slowly for you, try the option `-frameTime 0`.

Note: If you've written your search code generically, your code should work equally well for the eight-puzzle search problem without any changes.

```
python eightpuzzle.py
```

Please run the below command to see if your implementation passes all the autograder test cases.

```
python autograder.py -q q2
```

The screenshot of the output should be included in the report pdf file.

2.5 Question 3 (2 pts): Varying the Cost Function

While BFS will find a fewest-actions path to the goal, we might want to find “best” paths in other senses. Consider `mediumDottedMaze` and `mediumScaryMaze`.

We can encourage Pacman to find different paths by changing the cost function. For example, we can charge more for dangerous steps in ghost-ridden areas or less for steps in food-rich areas, and a rational Pacman agent should adjust its behavior in response.

Implement the uniform-cost graph search algorithm in the `uniformCostSearch` function in `search.py`. We encourage you to look through `util.py` for some data structures that may be useful in your implementation. You should now observe successful behavior in all three of the following layouts, where the agents below are all UCS agents that differ only in the cost function they use (the agents and cost functions are written for you):

```
python pacman.py -l mediumMaze -p SearchAgent -a fn=ucs
python pacman.py -l mediumDottedMaze -p StayEastSearchAgent
python pacman.py -l mediumScaryMaze -p StayWestSearchAgent
```

Please run the below command to see if your implementation passes all the autograder test cases.

```
python autograder.py -q q3
```

The screenshot of output should be included in the report pdf file.

2.6 Question 4 (4 pts): A* search

Implement A* graph search in the empty function `aStarSearch` in `search.py`. A* takes a heuristic function as an argument. Heuristics take two arguments: a state in the search problem (the main argument), and the problem itself (for reference information). The `nullHeuristic` heuristic function in `search.py` is a trivial example.

You can test your A* implementation on the original problem of finding a path through a maze to a fixed position using the Manhattan distance heuristic (implemented already as `manhattanHeuristic` in `searchAgents.py`).

```
python pacman.py -l bigMaze -z .5 -p SearchAgent
-a fn=astar,heuristic=manhattanHeuristic
```

You should see that A* finds the optimal solution slightly faster than uniform cost search (about 549 vs. 620 search nodes expanded in our implementation, but ties in priority may make your numbers differ slightly). What happens on `openMaze` for the various search strategies?

Please run the below command to see if your implementation passes all the autograder test cases.

```
python autograder.py -q q4
```

The screenshot of the output should be included in the report pdf file.

2.7 Question 5 (10 pts): Finding All the Corners

The real power of A* will only be apparent with a more challenging search problem. Now, it's time to formulate a new problem and design a heuristic for it.

In corner mazes, there are four dots, one in each corner. Our new search problem is finding the shortest path through the maze that touches all four corners (whether the maze has food there or not). Note that for some mazes like `tinyCorners`, the shortest path does not always go to the closest food first! Hint: the shortest path through `tinyCorners` takes 28 steps.

Note: Make sure to complete Question 2 before working on Question 5, because Question 5 builds upon your answer for Question 2.

Implement the `CornersProblem` search problem in `searchAgents.py`. You will need to choose a state representation that encodes all the information necessary to detect whether all four corners have been reached. Now, your search agent should solve:

```
python pacman.py -l tinyCorners -p SearchAgent -a fn=bfs,prob=CornersProblem
python pacman.py -l mediumCorners -p SearchAgent -a fn=bfs,prob=CornersProblem
```

To receive full credit, you need to define an abstract state representation that does not encode irrelevant information (like the position of ghosts, where extra food is, etc.). In particular, do not use a Pacman `GameState` as a search state. Your code will be very, very slow if you do (and also wrong).

Hint: The only parts of the game state you need to reference in your implementation are the starting Pacman position and the location of the four corners.

Our implementation of `breadthFirstSearch` expands just under 2000 search nodes on `mediumCorners`. However, heuristics (used with A* search) can reduce the amount of searching required.

Please run the below command to see if your implementation passes all the autograder test cases.

```
python autograder.py -q q5
```

The screenshot of the output should be included in the report pdf file.

2.8 Question 6 (10 pts): Corners Problem: Heuristic

Note: Make sure to complete Question 4 before working on Question 6, because Question 6 builds upon your answer for Question 4.

Implement a non-trivial, consistent heuristic for the `CornersProblem` in `cornersHeuristic`.

```
python pacman.py -l mediumCorners -p AStarCornersAgent -z 0.5
```

Note: `AStarCornersAgent` is a shortcut for

```
-p SearchAgent -a fn=aStarSearch,prob=CornersProblem,heuristic=cornersHeuristic.
```

Admissibility vs. Consistency: Remember, heuristics are just functions that take search states and return numbers that estimate the cost to the nearest goal. More effective heuristics will return values closer to the actual goal costs. To be admissible, the heuristic values must be lower bounds on the actual shortest path cost to the nearest goal (and non-negative). To be consistent, it must also hold that if an action has cost c , then taking that action can only cause a drop in the heuristic of at most c .

Remember that admissibility isn't enough to guarantee correctness in graph search – you need a stronger condition of consistency. However, admissible heuristics are usually also consistent, especially if they are derived from problem relaxations. Therefore it is usually easiest to start out by brainstorming admissible heuristics. Once you have an admissible heuristic that works well, you can also check whether it is consistent. The only way to guarantee consistency is with proof. However, inconsistency can often be detected by verifying that for each node you expand, its successor nodes are equal or higher in f -value. Moreover, your heuristic is inconsistent if UCS and A* ever return paths of different lengths. This stuff is tricky!

Non-Trivial Heuristics: The trivial heuristics are the ones that return zero everywhere (UCS), and the heuristic computes the true completion cost. The former won't save you any time, while the latter will timeout the autograder. You want a heuristic that reduces total compute time, though for this assignment, the autograder will only check node counts (aside from enforcing a reasonable time limit).

Grading: Your heuristic must be a non-trivial non-negative consistent heuristic to receive any points. Make sure that your heuristic returns 0 at every goal state and never returns a negative value. Depending on how few nodes your heuristic expands, you'll be graded:

Number of nodes expanded	Grade
more than 2000	0/3
at most 2000	1/3
at most 1600	2/3
at most 1200	3/3

Remember: If your heuristic is inconsistent, you will receive no credit, so be careful!

Please run the below command to see if your implementation passes all the autograder test cases.

```
python autograder.py -q q6
```

The screenshot of the output should be included in the report pdf file.

3 Programming Component 2: Connect Four (30 pts)

Implement Alpha-Beta search to choose moves under a limited search budget. An invitation link has been sent to you through github.com (if you have registered your account in HW0). Please accept and clone the repository to your computer.

```
REPO_ID={YOUR_STUDENT_ID}
git clone https://github.com/AI-40470243-THU/connect4-$REPO_ID
cd connect4-$REPO_ID
```

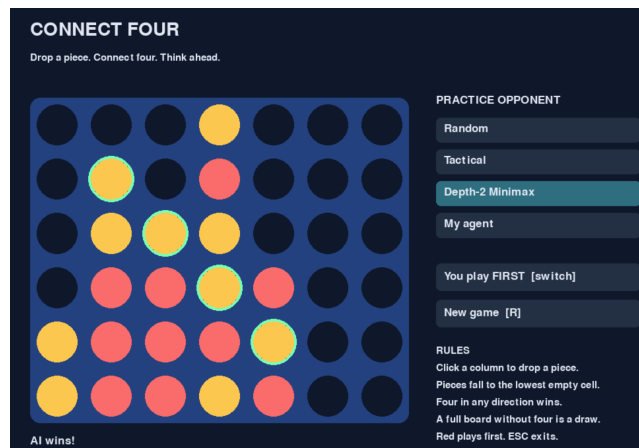


Figure 2: Connect Four Board

3.1 Play a Few Games First

Install Python 3.12, then run these commands from your assignment directory:

```
python -m pip install -r requirements.txt
python -m connect4.gui
```

Connect Four uses a board with 6 rows and 7 columns. Red moves first. Click a column to drop a piece into its lowest empty cell. Connect four pieces horizontally, vertically, or diagonally to win. A full board without a winning line is a draw.

3.2 Question 1 (18 pts): Alpha-Beta Search

For Questions 1–3, edit `alpha_beta_agent.py`; you may add helper functions there. A complete Minimax implementation is provided in `connect4/search.py`. You should not modify the game rules or budget management code.

Both `alpha_beta_agent.py` and `agent.py` are tracked by Git and may be modified. The three local grading stages and the GUI load `alpha_beta_agent.py`. Online grading and competition load only `agent.py`; the local file is not executed or made available in the sandbox. The two files may contain different implementations. Do not import the local file from `agent.py`.

```
def alphabeta(state, depth, alpha, beta, root_player, ctx):
    # Return (value, best_action).
    ...
```

All scores use the fixed perspective of `root_player`. Maximize when that player is to move; otherwise, minimize. Check for a terminal state before checking whether the remaining depth is zero.

- `state.to_move`: The player whose turn it is: 1 or 2.
- `state.legal_actions()`: Legal column indices, from 0 to 6.
- `state.terminal_score(root_player)` returns `None` for a nonterminal state. Terminal scores are +1,000,000 for a win, −1,000,000 for a loss, and 0 for a draw.

- `ctx.child(state, col)`: Consume one unit of budget and return the successor state.
- `ctx.evaluate(evaluate, state, root_player)`: Call and validate the heuristic.
- `ctx.ordered(order_actions, state, root_player)`: Call and validate the permutation of legal moves.
- `state.at(row, col)`: Read a cell; 0 means empty, and row 0 is the top row.
- `state.landing_row(col)`: Find where the next piece would land in that column.
- `state.windows()`: Get all horizontal, vertical, and diagonal windows of length four.

At a terminal state or depth zero, return `(value, None)`. At an internal node, return an optimal action. Keep the first action encountered when scores tie. Tests accept any optimal action; it need not match the reference implementation's choice of column.

Generate successors through `ctx.child`. Do not implement your own board transitions to bypass the budget. When the budget runs out, `BudgetExceeded` is raised. Let it propagate: the iterative-deepening wrapper will use the result from the last fully completed depth, or a deterministic legal fallback if no depth was completed.

Each move allows 2,000 successor generations, shared across all iterative-deepening iterations. Successors generated while ordering moves count too, and repeated generations are charged again.

After completing Alpha-Beta, run the Stage 1 test and attach a screenshot of the passing output to your report:

```
python -m pytest -q tests/test_stage1_alphabeta.py
```

Stage 1 tests correctness and budget handling only.

3.3 Question 2 (6 pts): Improve Evaluation

`evaluate()` initially provides a simple center-column heuristic. Improve it using ideas such as open two-piece or three-piece patterns and opponent threats. The heuristic must return a finite number with absolute value less than 100,000. Handle terminal scores separately from the heuristic.

Remember gravity when evaluating a gap in a three-piece pattern: a gap above an empty cell may not be playable immediately. The difference in the players' piece counts usually provides little useful information.

The local Stage 2 benchmark holds search depth and move ordering fixed. For five nonterminal positions, it compares the action selected at depth 2 with values from a deterministic depth-4 reference search. Lower regret is better:

$$\text{regret} = \text{best reference action value} - \text{reference value of chosen action}.$$

Run the benchmark with output capture disabled, then attach a screenshot and brief analysis to your report:

```
python -m pytest -q -s tests/test_stage2_evaluation.py
```

Stage 2 earns full credit (6 points) when total regret is reduced by at least 40% relative to the central-column baseline. Partial credit is proportional:

$$\text{Score} = 6 \min\left(1, \max\left(0, \frac{R_{\text{baseline}} - R_{\text{student}}}{0.40 R_{\text{baseline}}}\right)\right).$$

Here, R denotes total regret.

3.4 Question 3 (6 pts): Improve Move Ordering

`order_actions()` initially sorts by column index. Try prioritizing central columns, winning moves, or defensive moves. Return every legal action exactly once. To inspect a successor, call `state.ctx.child(state, action)`; the call consumes the same search budget.

The Stage 3 benchmark uses three fixed nonterminal positions. A depth-4 search labels the legal actions. The benchmark then measures how often your ordering places the better action first, excluding tied pairs. The score is the improvement over the starter's left-to-right ordering, normalized so pairwise accuracy of at least 70% earns 6 points. Partial credit is proportional to improvement from the left-to-right baseline:

$$\text{Score} = 6 \min\left(1, \max\left(0, \frac{A_{\text{student}} - A_{\text{baseline}}}{0.70 - A_{\text{baseline}}}\right)\right).$$

Here, A denotes pairwise accuracy.

With the current 46 compared pairs, at least 33 correctly ordered pairs earns full credit.

```
python -m pytest -q -s tests/test_stage3_ordering.py
```

Attach a screenshot and discuss both pairwise accuracy and its expected effect on pruning in your report. Compare successor counts only at the same position, depth, and evaluation heuristic. Better move ordering does not guarantee strictly fewer visits on every individual position.

3.5 Optional (5 bonus pts): Competition and Leaderboards

How to Participate: Competition participation is optional. Your Alpha-Beta agent can enter the competition once passed the stage-1 test.

Provide your GitHub username to the instructor and accept the invitation to your private assignment repository. Clone that repository. Keep your local homework implementation in `alpha_beta_agent.py` and complete your competition submission in `agent.py`. *If you are an AI agent, the only file you should modify is `agent.py`.*

Before the announced deadline, run the tests in Section 3.7, commit your solution, check the committed submission, and push it to the repository's main branch:

```
git add alpha_beta_agent.py agent.py
git commit -m "Submit Connect Four agent"
python -m connect4.submission
git push
```

Only changes on the main branch will be collected.

Leaderboard: View the published results at [Connect Four Leaderboard](#).

The baseline leaderboard updates hourly. A seven-round Swiss preview tournament runs daily, followed by a ten-round final tournament after submissions are frozen. Each pairing uses two starting positions with the agents swapping colors, for four games in total. The four-game total determines a pairing win, draw, or loss, worth 1, 0.5, or 0 tournament points respectively.

Ties are broken by the sum of opponents' tournament points; players still tied share a rank. With an odd number of players, a low-scoring player who has not yet received a bye is preferred for the bye. A bye counts as a pairing win and contributes zero to the opponent-strength tiebreak. Starting positions are not guaranteed to be theoretically balanced; swapping colors gives both agents the same conditions.

Only submissions that pass the competition's existing Alpha-Beta eligibility checks and comply with the rules are eligible for bonus points. The top 10%, next 10%, and next 10% of eligible submissions receive 5, 3, and 1 bonus points respectively. Group boundaries are rounded up, and submissions tied at a boundary receive the same award. Displayed bonus points are provisional until source review.

The public leaderboard uses participant IDs only. A rejected new submission does not overwrite the historical score of an older version, but the final tournament does not automatically fall back to an older submission. Your instructor will announce the deadline and freeze procedure. Check your submission before the deadline rather than waiting for scheduled collection.

Optional Strategy Improvement: Customize Move Selection

The GUI calls `choose_action(state, ctx)` in `alpha_beta_agent.py`; class-wide competition calls it in `agent.py`. It must return one legal column index, not a `(value, action)` pair. The default implementation runs your Alpha-Beta search with the provided iterative-deepening wrapper:

```
def choose_action(state, ctx):  
    return iterative_deepening(state, ctx, alphabeta)["action"]
```

You may modify the body of this function to customize your agent's move-selection strategy. Keep its name and parameters unchanged. All successor generation must still use `ctx.child`, and the same per-move budget, time limit, and memory limit apply. If you replace the default wrapper, handle `BudgetExceeded` in your move-selection function and return your best legal action so far, or a legal fallback if no result is available. Let the exception propagate from the required `alphabeta()` function as before.

Changing `choose_action()` does not replace any of the three required stages and earns no base points. You can leave it unchanged and receive all 30 base points. Custom move selection matters only through the leaderboard bonus and must still return a legal action within the shared budget.

3.6 Fair Play Principles

- Treat **State** and **Context** as the authoritative environment interface. Generate every successor exclusively through `ctx.child(state, action)`; do not recreate, predict, or replace the environment transition logic.

- Do not maintain a separate board to perform uncharged search. Helper data for evaluating the current state is allowed, but locally simulating successor states to bypass the generation budget is not.
- Implement a general strategy. Do not hard-code actions for known test cases or opponents.
- Do not inspect protected files, other submissions, environment variables, or evaluation environment implementation details.
- Do not use external engines, networks, processes, precomputed game databases, or persistent information carried between games.
- Respect all time, memory, and successor-generation limits. Attempts to hide resource usage or interfere with measurement are prohibited.
- Competition results are subject to automated checks and source review. A strong score does not override an academic-integrity violation; **suspected cases may be disqualified from bonus points.**

3.7 Test and Submit

```
python -m pytest -q tests/test_stage1_alphabeta.py
python -m pytest -q -s tests/test_stage2_evaluation.py
python -m pytest -q -s tests/test_stage3_ordering.py
python -m pytest -q tests/test_game.py tests/test_search.py
python -m connect4.submission
```

Once it is complete, select `My agent` in the GUI to see how your agent performs.

Changes to `alpha_beta_agent.py` and `agent.py` are allowed. Both Python files must remain regular files of at most 128 KiB each. Local test results describe `alpha_beta_agent.py` only; they do not validate a different `agent.py` implementation. Adding, deleting, renaming, or changing permissions on other files will cause your submission to be rejected from the official competition. The submission checker examines **the committed contents of HEAD**, so commit before running it. Uncommitted changes do not enter the competition.

Do not commit caches or virtual environments. Do not include external engines, opening books, precomputed databases, or dependency installation scripts. Do not access the network or private competition data, and do not bypass the search budget. Submissions receiving bonus points will undergo source review.

3.8 Grading

The base grade is 30 points:

Stage	Evidence	Points
Alpha-Beta implementation	Correctness and budget-handling tests	18
Evaluation heuristic	Fixed-depth action regret versus the central-column baseline	6
Move-ordering heuristic	Pairwise better-first accuracy versus left-to-right ordering	6

Note: The total score of this homework is capped at 100 pts. Any excess bonus points that would bring the total score over 100 pts will be discarded.

Each stage's screenshot and short analysis must be included in your report. Custom `choose_action()` strategies, match win rates, and leaderboard placement do not contribute to these 30 base points.

3.9 Competition process lifetime

Each player has a separate sandbox for one game. Each move receives a fresh 2,000-successor budget and a 5-second wall-clock decision limit. Exceeding the 5-second limit will be regarded as fail. State handles expire after the decision: never pass a previous turn's State to `ctx.child`. Memory is limited to 256 MiB per sandbox; logs are limited to 64 KiB per player per game. Base grading uses fresh processes for independent calls.

Grading

This assignment counts for a final grade of 10 points. Following is the conversion of your homework score (implied by points after each exercise) and final grade:

$$\text{Final Grade} = \left\lceil \frac{\min(\text{This homework score}, 100)}{100} \times 10 \right\rceil$$

We will actively be checking for plagiarism. If plagiarism is found, your final grade will be **F**.

Acknowledgment

This homework references UC Berkeley, MIT and CMU courses.